BCD(Decimal Codificado en Binario – Binary Coded Decimal): es la representación de los números decimales (0 al 9) escritos en sistema numérico binario.

CONSTRUCCIÓN DE TABLA BINARIA, independiente de la cantidad de variables

No.	2^7=128	2^6=64	2^5=32	2^4=16	2^3=8	2^2=4	2^1=2	2^0=1
Decimales				**E**	**D**	**C**	**B**	**A**
0				0	0	0	0	0
1				0	0	0	0	1
2				0	0	0	1	0
3				0	0	0	1	1
4				0	0	1	0	0
5				0	0	1	0	1
6				0	0	1	1	0
7				0	0	1	1	1
8				0	1	0	0	0
9				0	1	0	0	1
10				0	1	0	1	0
11				0	1	0	1	1
12				0	1	1	0	0
13				0	1	1	0	1
14				0	1	1	1	0
15				0	1	1	1	1



Importante: Cada columna tiene una ponderación de 2^x, lo que quiere decir que en esa columna se colocarán '0' y '1' alternados de a **x**, comenzando con '0'.

Columna A tiene una ponderación de 1, lo que quiere decir que en esa columna se colocarán '0' y '1' alternados de a uno, comenzando con '0'.

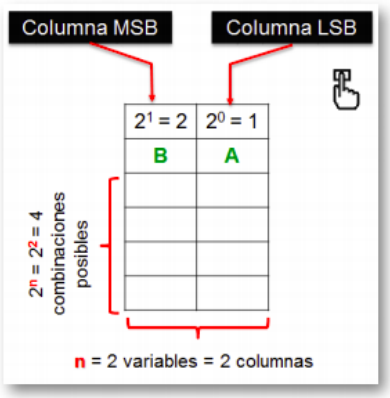
Columna D tiene una ponderación de 8, lo que quiere decir que en esa columna se colocarán '0' y '1' alternados de a ocho, comenzando con '0'.

Variables y Combinaciones para crear una tabla

Para crear una tabla se utiliza el sistema numérico binario, se le asigna a cada variable de la función una columna y por medio de la siguiente fórmula se obtiene el número de combinaciones que tendrá la tabla.

2^n = Número de combinaciones posibles

n = Cantidad de variables de la función



MSB: bit más significativo

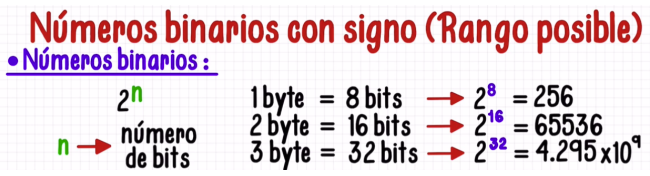
LSB: bit menos significativo

Columna A tiene una ponderación de 1, lo que quiere decir que en esa columna se colocarán '0' y '1' alternados de a uno, comenzando con '0'.

Columna B tiene una ponderación de 2, lo que quiere decir que en esa columna se colocarán '0' y '1' alternados de a dos, comenzando con '0'.

Los números binarios con signo permiten representar positivos y negativos, y su rango depende del número de bits disponibles. La representación más común es el complemento a dos, donde el primer bit indica el signo y el rango es de $-2^{(n-1)}$ a $2^{(n-1)}-1$.
Rango de los números binarios con signo (positivos y negativos)

1. **Representación binaria con signo**
   - Se usa un **bit de signo**:
     - `0` → número positivo.
     - `1` → número negativo.
2. **Formas comunes de representar enteros con signo**
   - **Signo y magnitud:** un bit para el signo y el resto para la magnitud.
   - **Complemento a uno:** negativos se representan invirtiendo los bits del positivo.
   - **Complemento a dos (la más usada):** negativos se obtienen invirtiendo los bits y sumando 1.
3. **Rango de valores según n bits (en complemento a dos):**
   - Valor mínimo: -2^{n-1}
   - Valor máximo: $2^{n-1}-1$
   - Ejemplo con 8 bits: rango = **–128 a +127**.
4. **Ejercicios típicos**
   - Convertir un número decimal (positivo o negativo) a binario con signo.
   - Identificar el rango de un número dado el tamaño en bits.
   - Pasar de binario con signo a decimal.



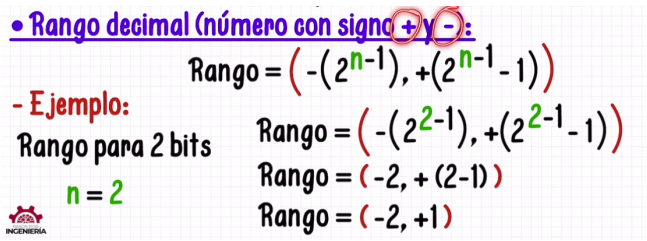
Que significa:
Tenemos diferentes maneras de representar los números decimales, y podemos ocupar tipos como:
1 byte = 8bits, es decir tenemos 256 maneras de representar números esto equivale, a que puedo representar números de 0..255
2 byte = 16bits, es decir tenemos 65536 maneras de representar números esto equivale, a que puedo representar números de 0..65535

Por tanto, si por ejemplo quiero representar un número 300, pues con 1byte no podría representarlo, estoy obligado a usar 2 byte

Abajo una tabla con los más reconocidos en c++

n: Es el número de bits, o número de variables binarias

Tamaño (bytes)	Tipo estándar C++	Notas
1 byte (8 bits)	char / signed char / unsigned char	Usado para caracteres y enteros pequeños
2 bytes (16 bits)	short / unsigned short	Enteros cortos
3 bytes (24 bits)	❌ No existe tipo nativo	Se simula con struct o librerías, o usando int32_t y limitando rango
4 bytes (32 bits)	int / unsigned int / float	Enteros comunes y punto flotante simple
8 bytes (64 bits)	long long / unsigned long long / double	Enteros largos y punto flotante doble
16 bytes (128 bits)	No estándar, pero algunos compiladores ofrecen __int128 o long double	Depende del compilador (no portable)

 Para buscar un rango de decimales con números tenemos la función de encima, y si por ejemplo tomamos que **n=2(solo con 2 bits)**, entonces el rango de número que podré representar con signo va a ir desde **-2 hasta +1**, es decir se van a poder representar al **-2, -1, 0 y al 1(**es decir 4 combinaciones posibles **2^n)**	 Este es otro ejemplo pero con **n=4(solo con 4 bits)**

Rango de números: Todos los valores independientes posibles que se pueden representar en n cantidad de bits.

Representar números **Enteros sin signo(**Solo positivos**):** $0..2^n-1$

Ejemplo	
4bits = 2^3=8	0,1,2,3..7 (111)
8bits = 2^8=8	0,1,2,3..255 (11111111)
Para representar por ejemplo el $(300)_{10}$ no alcanza con 8bits, por tanto tendríamos que ocupar otro bits, y como los cálculos en java se usan de bytes en bytes, pues se utilizará 2bytes=16bits, porque con 1bytes=8bits, no es suficiente.	

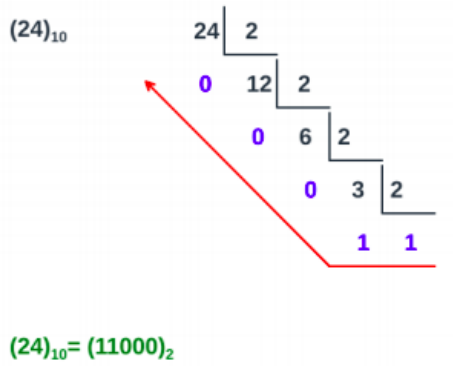
Representar números **Enteros con signo(**Negativos y positivos**):** $(-(2^{n-1}); 2^{n-1}-1)$

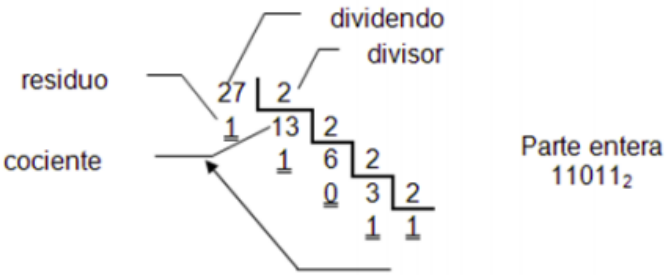
Ejemplo	**$(-(2^{n-1}); 2^{n-1}-1)$**	
2bits = $(-(2^{2-1}); 2^{2-1}-1) = (-2; +1)$	-2,-1, 0,1	(XX)
4bits = $(-(2^{4-1}); 2^{4-1}-1) = (-8; +7)$	-8,-7…. +7	(XXXX)
MSB: bit más significativo **1**: Negativo **0**: Positivo		

https://www.youtube.com/watch?v=RT89h-oko70

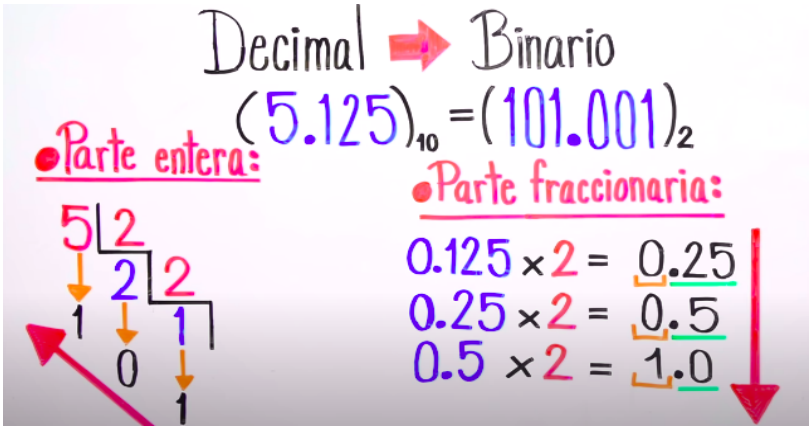
Conversión de Decimal a Binario

Para la conversión de la **parte entera**, se va dividiendo la cantidad decimal por 2 (divisiones sucesivas), tantas veces como sea posible, hasta que llegue a un cociente de 1. Luego, se toma el último "Cociente" y todos los "Residuos" comenzando con el último (**bit más significativo** - **MSB**) hasta llegar al primero (**bit menos significativo** – **LSB**).





Para la conversión de la **parte fraccionaria** se multiplica por la base (2) sucesivamente hasta la cantidad de dígitos que se requieran(donde queda un número cercano a cero ejemp. **.003**), se toman para la respuesta los números enteros de cada producto y lo que sobra lo sigue multiplicando por 2.



Conversión de Binario a Decimal

101,001 = $1*2^2+0*2^1+1*2^0+0*2^{-1}+0*2^{-2}+1*2^{-3}$

= **4 + 0 + 1 + 0 + 0 + ⅛**

= **5 + 0.125**

= **5.125**

Para la **parte entera** multiplique cada bit por 2 elevado a la potencia consecutiva (comenzando con 2^0), a esto se le denomina técnicamente "ponderar"; después de ello realice la suma de todos los resultados y el número resultante será el equivalente al sistema decimal.

Para la **parte fraccionaria**, tome los dígitos de izquierda a derecha con exponentes negativos comenzando con 2^{-1} y multiplique por cada bit que está en esa posición y finalmente sume los resultados de la parte fraccionaria.

Representar números enteros con signos

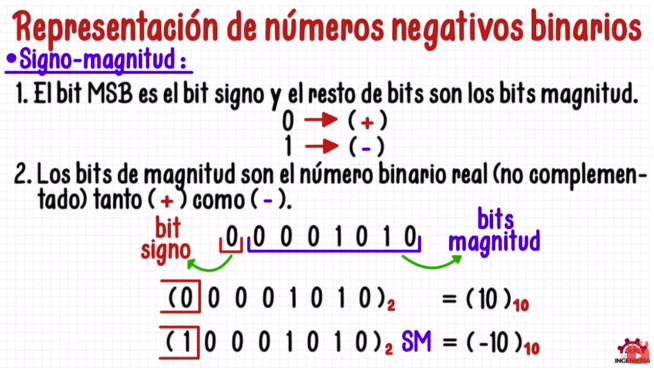
Formas comunes o convenios de representar números enteros con signos(Existen 3 formas):

- **Signo y magnitud:** un bit para el signo y el resto para la magnitud.
- **Complemento a uno:** negativos se representan invirtiendo los bits del positivo.
- **Complemento a dos (la más usada):** negativos se obtienen invirtiendo los bits y sumando 1.

Representación de números decimales escritos en sistema numérico binario
BCD(Decimal Codificado en Binario – Binary Coded Decimal): es la representación de los números decimales (0 al 9) escritos en sistema numérico binario.

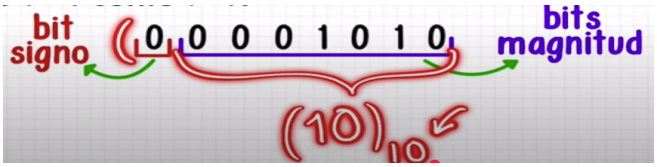
Signo y magnitud(SM)

Signo y magnitud(SM): La representación signo-magnitud usa **un bit para el signo** y los **demás para la magnitud** del número, pero tiene el inconveniente de representar dos ceros distintos y complicar la aritmética.
https://www.youtube.com/watch?v=gmlwYa7QED8&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=14



Ejemplo práctico (4 bits)

Decimal	Binario signo-magnitud
+3	0011
–3	1011
+7	0111
–7	1111
+0	0000
–0	1000



1. **¿Qué es la representación signo-magnitud?**
   - Se reserva **el bit más significativo (MSB)** para el signo:
     - 0 → número positivo
     - 1 → número negativo
   - El resto de los bits representan la **magnitud (valor absoluto)**.

Ejemplo en 4 bits:

- **0**101 → +5 (**0** → número positivo)
- **1**101 → –5 (**1** → número positivo)

2. Ventajas y desventajas

✔️ **Ventaja**: Fácil de entender, ya que es muy parecido a la notación decimal con signo.
❌ **Desventaja**: La duplicidad del 0 y la dificultad para hacer operaciones aritméticas (suma y resta).

Formas o convenios de representar números enteros con signos:

Representación de números decimales escritos en sistema numérico binario
BCD(Decimal Codificado en Binario – Binary Coded Decimal): es la representación de los números decimales (0 al 9) escritos en sistema numérico binario.

Signo y magnitud(SM): Nos permite codificar un número entero con signo en binario.
- Escribir el número **binario real** (**no complementado**)

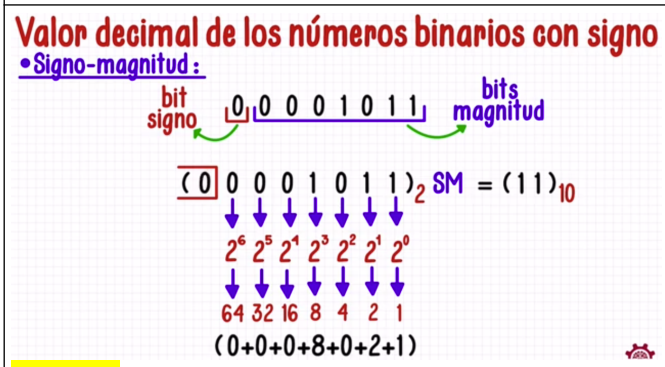
- Esto nos otorga **1 bit(MSB)** para el signo y **el resto de bits** para la magnitud.
- Los bits de magnitud son el número binario real(no complementado).

$(10)_{10}$ = (**0**0001010)$_2$SM (**Positivo**)

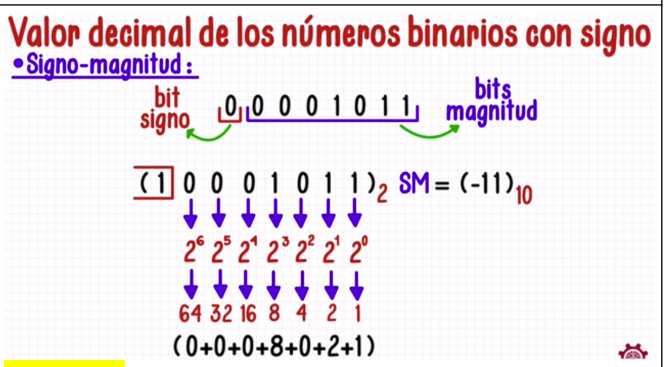
$(-10)_{10}$ = (**1**0001010)$_2$SM (**Negativo**)

Convertir de Binario SM a Decimal por

Convertir de Binario SM a Decimal: En signo-magnitud, convertir de binario a decimal es tan simple como leer el primer bit como signo y los demás como valor absoluto.
https://www.youtube.com/watch?v=9flkTGExUJQ&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=17



Positivo



Negativo

SIGNO MAGNITUD - CONVERTIR de BINARIO A DECIMAL (NÚMEROS BINARIOS NEGATIVOS Y POSITIVOS)

Conversión de **binario a decimal** en **Signo-Magnitud**

1. **Reglas básicas**
   1. El **bit más significativo (MSB)** indica el signo:
      - 0 → positivo
      - 1 → negativo
   2. Los demás bits representan la **magnitud (valor absoluto)**.
   3. Se convierte la magnitud a decimal normalmente, y luego se aplica el signo.

2. **Ejemplo paso a paso (4 bits)**

Ejemplo 1: **0**101
- MSB = **0** → positivo.
- Magnitud = 101 = 5 en decimal.
- Resultado: **+5**.

Ejemplo 2: 1101
- MSB = **1** → negativo.
- Magnitud = 101 = 5 en decimal.
- Resultado: **–5**.

Ejemplo 3: 1000
- MSB = **1** → negativo.
- Magnitud = 000 = 0.
- Resultado: **–0** (problema del doble cero).

3. **Ejemplo con 8 bits**

Ejemplo 4: 00010101
- MSB = **0** → positivo.
- Magnitud = 0010101 = 21.
- Resultado: **+21**.

Ejemplo 5: 10010101
- MSB = **1** → negativo.
- Magnitud = 0010101 = 21.
- Resultado: **–21**.

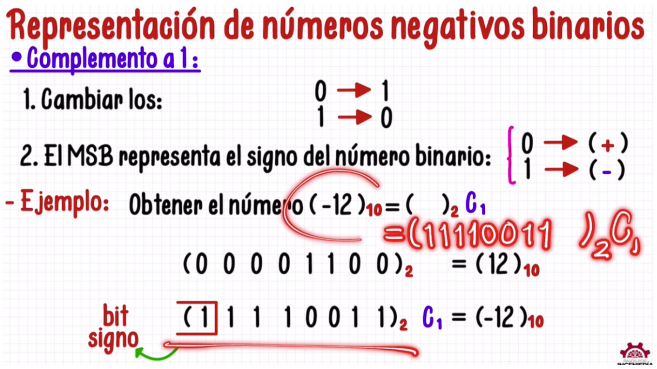
4. **Resumen del procedimiento**
   1. Revisar el **primer bit** (signo).
   2. Convertir los demás bits a decimal.
   3. Aplicar el signo correspondiente.

No.	2^7=128	2^6=64	2^5=32	2^4=16	2^3=8	2^2=4	2^1=2	2^0=1	
$(10)_{10}$	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**0**	
SM	**NO**	0	0	**0**	**8**	**0**	2	**0**	**10**

$(10)_{10}$	**1**	**0**	**0**	**0**	**1**	**0**	**1**	**0**	
SM	**NO**	0	0	**0**	**8**	**0**	2	**0**	**-10**

Complemento a uno(C_1)

Complemento a uno: El complemento a 1 representa **números negativos invirtiendo todos los bits** del positivo, pero presenta el inconveniente de tener dos ceros (positivo y negativo).
https://www.youtube.com/watch?v=LyaysLyiyzE&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=15



Números binarios negativos (Complemento a 1)
1. **¿Qué es el complemento a 1?**
   - Es un método para representar números **negativos en binario**.
   - El bit más significativo (**MSB**) indica el signo:
     - 0 → positivo
     - 1 → negativo
   - Para obtener el negativo de un número:
     👉 **Se invierten todos los bits** del número positivo.

Ejemplo con 4 bits:
- +5 = **0**101 (los positivos no se invierten)
- –5 = **1**010 (invirtiendo todos los bits, solo para el caso de negativos).

2. Rango de representación
Con **n bits**:
- Valores positivos: de 0 hasta $2^{n-1}-1$.
- Valores negativos: de $-(2^{n-1}-1)$ hasta –0.

Ejemplo con 4 bits:
- Positivos: **0**000 (0) hasta **0**111 (+7).
- Negativos: **1**111 (–0) hasta **1**000 (–7).

3. Problema del "doble cero"
- Igual que en **signo-magnitud**, aparecen **dos representaciones de cero**:
  - 0000 = +0
  - 1111 = –0

4. Ventajas y desventajas
✔️ **Ventaja**: La resta puede hacerse como suma (más simple que en signo-magnitud).
❌ **Desventaja**: Sigue existiendo el **doble cero** y complica un poco la aritmética en hardware.

Complemento a uno(c1):

Si el número es **positivo**: (igual que el **SM**)

- Escribir el número **binario real** (no complementado)
- Esto nos otorga **1 bit(MSB)** para el signo 0=Positivo
- E**l resto de bits** para la magnitud se representa en binario natural

$(4)_{10}$ = (**0**0000100)$_2$C1 (**Positivo**)

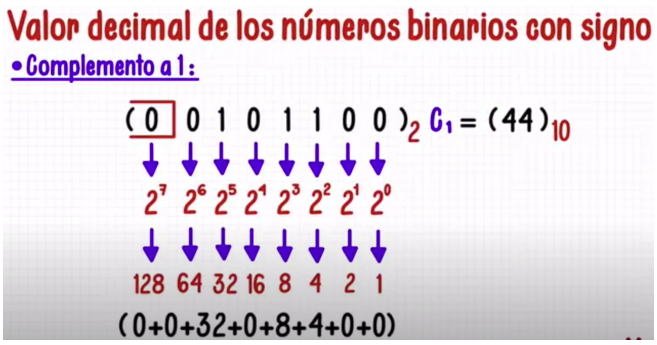
Si el número es **negativo**:
- Escribir el número **binario real** (no complementado)
- Esto nos otorga **1 bit(MSB)** para el signo. 1=negativo
- E**l resto de bits** se invierten 1-0, 0-1

$(4)_{10}$ = (**0**0000100)$_2$ (**real 4 decimal positivo**)
$(-4)_{10}$ = (**1**1111011)$_2$C1 (**Negativo**)

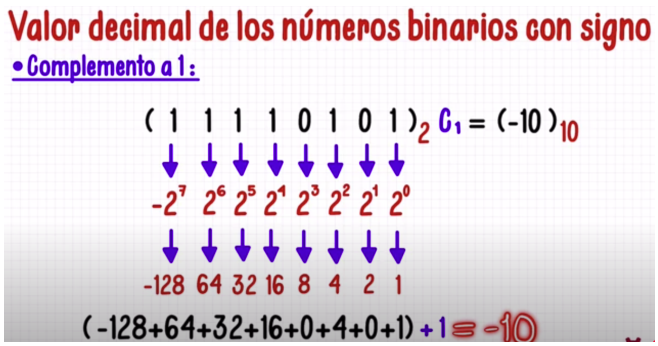
$(10)_{10}$ = (**0**0001010)$_2$ (**real 10 decimal positivo**)
$(-10)_{10}$ = (**1**1110101)$_2$C1 (**Negativo**)

Convertir de Binario C1 a Decimal por

Convertir de Binario C1 a Decimal por: En Complemento a 1, los números negativos se interpretan invirtiendo los bits y colocando el signo (lo que genera el inconveniente de tener dos ceros: +0 y –0). https://www.youtube.com/watch?v=VmGcPQtvPok&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=18



positivo



Negativo:
Nota: Cuando es negativo La base MSB es -2 y ademas se suma +1

COMPLEMENTO A 1 - CONVERTIR de BINARIO A DECIMAL (NÚMEROS BINARIOS NEGATIVOS Y POSITIVOS)
Conversión de binario a decimal en **Complemento a 1**
1. **Reglas básicas**
   1. **Si el MSB (bit más significativo) es 0 → número positivo**
      - Se convierte normalmente a decimal.
   2. **Si el MSB es 1 → número negativo**
      - El número está en **Complemento a 1**.
      - Para obtener el valor en decimal:
        - **Invertir todos los bits** (volver a positivo).
        - Convertir a decimal.
        - Colocar el signo negativo.

3. **Ejemplo en 8 bits**
Ejemplo 4: 00010101
- MSB = 0 → positivo.
- Magnitud = 21.
- Resultado: **+21**.

Ejemplo 5: 11101010
- MSB = 1 → negativo.
- Invertir bits: 11101010 → 00010101.
- Magnitud = 21.
- Resultado: **–21**.

4. **Resumen del procedimiento**
   1. Verificar el **primer bit**.
   2. Si es 0: convertir normalmente a decimal.
   3. Si es 1: invertir los bits, convertir y poner

2. **Ejemplo en 4 bits**

Ejemplo 1: **0**101

- MSB = **0** → **positivo**.
- Magnitud = 101 = 5.
- Resultado: **+5**.

Ejemplo 2: **1010**

- MSB = **1** → **negativo**.
- **Invertir** bits: 1**010** → 0**101**.
- Magnitud = 5.
- Resultado: **–5**.

Ejemplo 3: **1**111

- MSB = **1** → negativo.
- Invertir bits: **1**111 → 0**000**.
- Magnitud = 0.
- Resultado: **–0** (problema del doble cero).

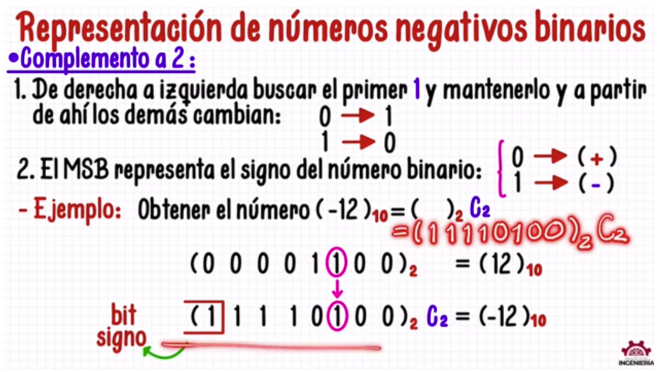
signo negativo.

No.	2^7=128	2^6=64	2^5=32	2^4=16	2^3=8	2^2=4	2^1=2	2^0=1	
$(4)_{10}$	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	
	0	0	0	**0**	**0**	**4**	0	**0**	**4**
$(-4)_{10}$	**1**	**1**	**1**	**1**	**1**	**0**	**1**	**1**	
	-128	64	32	**16**	**8**	0	2	**1**	**-4**

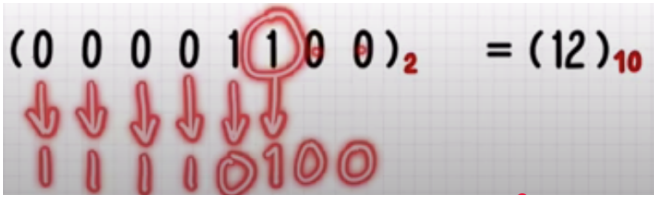
(−128+64+32+16+8+0+2+1)**+(1)** = **-4**

Complemento a dos (la más usada C_2):

El complemento a 2 es el sistema más usado porque evita el doble cero y simplifica las operaciones aritméticas en hardware. Porque una maquina lo puede calcular facilmente mediante el uso de inversores y un sumador https://www.youtube.com/watch?v=JQyN0D_2X_M&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=16



El primer paso: Encontrar el **primer 1 y mantenerlo,** y a partir de ahí los **demás cambian**



Números binarios negativos (Complemento a 2)

1. ¿Qué es el complemento a 2?

Es el **sistema estándar** para representar números enteros con signo en la mayoría de los lenguajes de programación y procesadores.

- El bit más significativo (**MSB**) indica el signo:
  - 0 → número positivo.
  - 1 → número negativo.
- Para obtener el negativo de un número: **Se invierten todos los bits (complemento a 1) y se suma 1**.

Ejemplo con 4 bits:

- +5 = 0101
- –5 = (1010 + 1) = 1011

2. Rango de representación

Con **n bits**:

- Valor mínimo: -2^{n-1}
- Valor máximo: $2^{n-1}-1$

Ejemplo con 4 bits:

- Mínimo = –8 (1000)
- Máximo = +7 (0111)
- Rango: **–8 a +7**

3. Ventajas principales

✔️ **Un solo cero** (se elimina el problema de +0 y –0).
✔️ La **suma y resta son naturales**, el hardware no necesita reglas especiales.
✔️ Es la **forma universal en computadoras** para manejar enteros con signo.

4. Ejemplo práctico (4 bits)

Decimal	Binario (C2)
+3	0011
–3	1101
+7	0111
–7	1001
–8	1000
0	0000

Complemento a dos(c2):

Si el número es **positivo**: (igual que el **SM**)
- Escribir el número **binario real** (no complementado)
- Esto nos otorga **1 bit(MSB)** para el signo 0=Positivo
- E**l resto de bits** para la magnitud se representa en binario natural

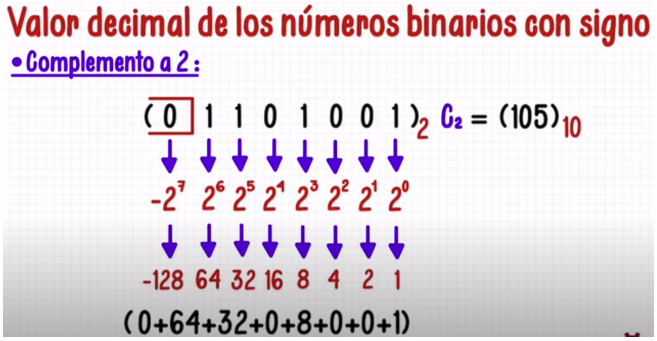
$(105)_{10}$ = (**0**1101001)$_2$ (**real 105 decimal positivo**)
$(105)_{10}$ = (**0**1101001)$_2$C1 (**Positivo**)

Si el número es **negativo.**
- Escribir el número **binario real positivo**(no complementado)
- De derecha a izquierda, encontrar el **primer 1** y mantenerlo, a partir de ahí los demás cambian. (se invierten 1-0, 0-1)
- El bit más significativo**(MSB)** para el signo. 1=negativo, 0=Positivo

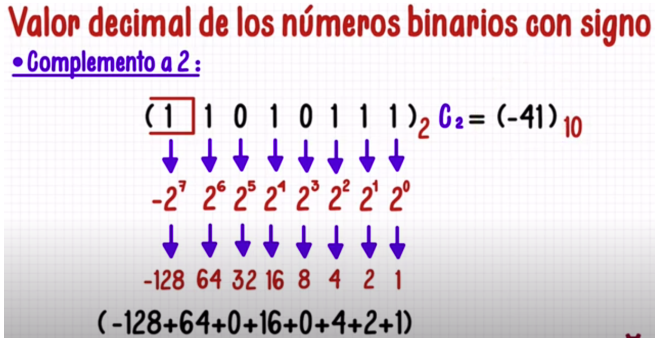
Ejemplo Negativo
$(12)_{10}$ = (**0**0001100)$_2$ (**real 12 decimal positivo**)
$(-12)_{10}$ = (**1**1110100)$_2$C2

Convertir de Binario C2 a Decimal

Convertir de Binario C2 a Decimal: En Complemento a 2, los números positivos se leen normalmente y los negativos se obtienen restando 2n o aplicando el proceso de inversión +1. Es el sistema más eficiente porque tiene un único cero y permite hacer aritmética de forma directa.
https://www.youtube.com/watch?v=DQwbtP-kiGU&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=19



Positivo
Nota: **La base MSB es -2**



Negativo
Nota: **La base MSB es -2**

COMPLEMENTO A 2 - CONVERTIR de BINARIO A DECIMAL (NÚMEROS BINARIOS NEGATIVOS Y POSITIVOS)

1. **Reglas básicas**
   1. **Si el MSB (bit más significativo) es 0 → número positivo**
      - Se convierte normalmente a decimal.
   2. **Si el MSB es 1 → número negativo**
      - El número está en **Complemento a 2**.
      - Para obtener el valor en decimal:
        - **Opción A** (**rápida**): interpretar normalmente y **restar** 2^n (donde n

3. **Ejemplo en 8 bits**

Ejemplo 4: **0**0010101
- MSB = **0** → **positivo**.
- Magnitud = 21.
- Resultado: **+21**.

Ejemplo 5: **1**1101011
- MSB = **1** → **negativo**.
- **Método A**: interpretar 11101011 = 235 → 235 – 256 = **–21**.
- **Método B**: invertir 11101011 → 00010100; sumar 1 → 00010101 = 21 → **–21**.

4. **Rango con n bits**

= número de bits). • **Opción B** (**paso a paso**): 1. Invertir todos los bits. 2. Sumar 1. 3. Convertir a decimal. 4. Colocar signo negativo. 2. **Ejemplo en 4 bits** **(2^n=16)** Ejemplo 1: **0**101 • MSB = **0** → **positivo**. • Magnitud = 5. • Resultado: **+5**. Ejemplo 2: **1**011 • MSB = **1** → **negativo**. • **Método A**: interpretar 1011 = 11 → 11 – 16 = **–5**. • **Método B**: invertir bits 1011 → 0100; sumar 1 → 0101 = 5 → **–5**. Ejemplo 3: 1000 • MSB = **1** → **negativo**. • **Método A**: interpretar **1**000 = 8 → 8 – 16 = **–8**. • Resultado: **–8** (el más negativo posible con 4 bits).	• Valor mínimo: -2^{n-1}. • Valor máximo: $2^{n-1}-1$. • Ejemplo: con 8 bits → de **–128 a +127**.

No.	$-2^7=-128$	$2^6=64$	$2^5=32$	$2^4=16$	$2^3=8$	$2^2=4$	$2^1=2$	$2^0=1$	
C2	**0**	**1**	**1**	**0**	**1**	**0**	**0**	**1**	
	0	64	32	**0**	**8**	**0**	0	**1**	**105**
$(-4)_{10}$	**1**	**1**	**1**	**1**	**0**	**1**	**0**	**0**	
C1	**-128**	64	32	**16**	**0**	4	0	**0**	**-12**

(−128+64+32+16+0+4+0+0) = **-12**

exceso K: donde normalmente K equivale a b^{n-1}- 1.

Comparación de representaciones de números binarios negativos

1. Conceptos generales

Representación	Cómo se obtiene el negativo	Problema principal	Ceros	Rango (n bits)
Signo-Magnitud	MSB=1 indica negativo; magnitud igual que el positivo	Doble cero, aritmética complicada	2	$-(2^{(n-1)}-1)\ldots+(2^{(n-1)}-1)$

Representación	Cómo se obtiene el negativo	Problema principal	Ceros	Rango (n bits)
Complemento a 1	Invertir todos los bits	Doble cero	2	$-(2^{(n-1)}-1) \dots +(2^{(n-1)}-1)$
Complemento a 2	Invertir bits + sumar 1	Ninguno relevante	1	$-2^{(n-1)} \dots +(2^{(n-1)}-1)$

---

2. Ejemplo en 4 bits

Decimal	Signo-Magnitud	C1 (Comp. a 1)	C2 (Comp. a 2)
+0	`0000`	`0000`	`0000`
–0	`1000`	`1111`	❌ (no existe)
+3	`0011`	`0011`	`0011`
–3	`1011`	`1100`	`1101`
+7	`0111`	`0111`	`0111`
–7	`1111`	`1000`	`1001`
–8	❌ (no existe)	❌ (no existe)	`1000`

---

3. Ejemplo en 8 bits

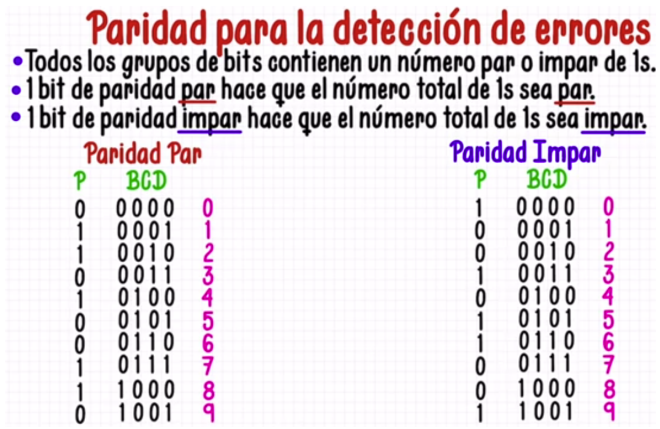
Decimal	Signo-Magnitud	C1 (Comp. a 1)	C2 (Comp. a 2)
+5	`00000101`	`00000101`	`00000101`
–5	`10000101`	`11111010`	`11111011`
+127	`01111111`	`01111111`	`01111111`
–127	`11111111`	`10000000`	`10000001`
–128	❌	❌	`10000000`

✅ **Conclusión:**

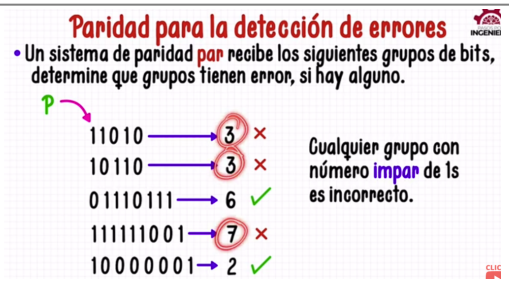
- **Signo-magnitud** y **complemento a 1** sufren el problema de tener **dos ceros**.
- **Complemento a 2** es el más eficiente, elimina el cero duplicado y extiende el rango negativo en 1 unidad.
- Por eso, **todas las computadoras modernas usan complemento a 2**.

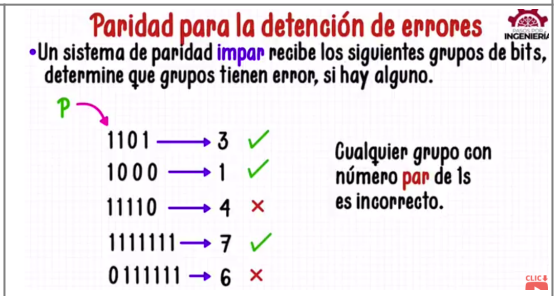
Paridad para la detección de errores https://www.youtube.com/watch?v=hEhp8qhFU90&list=PL46-B5QR6sHntg3995-PvwyzUGZu4epRU&index=20

Cuando trabajamos con sistemas binarios es común que en la transmisión de datos de bits a bits haya errores, es por eso que la detección de errores, como la paridad, o el método de Hamming, resultan útiles y es muy sencillo aplicarlos.

	Todos los grupos de bits contienen un número par o impar de 1s. 1bit de paridad par hace que el **número total de 1**s sea par 1bit de paridad impar hace que el **número total de 1**s sea impar

Paridad **https://es.wikipedia.org/wiki/C%C3%B3digo_Hamming**				
Paridad par Total de 1 sea par =0, sino 1	**BCD**		**Paridad impar** Total de 1 sea impar =0, sino 1	**BCD**
0	0000		1	0000
1	000**1**		0	000**1**
1	00**1**0		0	00**1**0
0	00**11**		1	00**11**
1	0**1**00		0	0**1**00
0	0**101**		1	0**101**
0	0**11**0		1	0**11**0
1	0**111**		0	0**111**
1	**1**000		0	**1**000
0	**1**00**1**		1	**1**00**1**
Esto hace que todo el código BCD este en #par, porque ahora al juntar P y BCD, los 1s, se hacen par. Ejemplo: 1 y 0**111** = **1**0**111** Por tanto, cuando hagamos una transmisión de datos de bits a bits podemos identificar si hay			Esto hace que todo el código BCD este en #impar, porque ahora al juntar P y BCD, los 1s, se hacen impar. Ejemplo: 1 y 0**11**0 = **1**0**110** Por tanto, cuando hagamos una transmisión de datos de bits a bits podemos identificar si hay errores, porque todos deben pasar como impar	

<table>
<tr><td>errores, porque todos deben pasar como par de 1s</td><td></td><td>de 1s</td></tr>
<tr><td colspan="3">Si tuviéramos que comunicar información en BCD, añadiríamos un bit de paridad, ya se si estamos en Paridad par, o Paridad impar o lo que nos convenga en el diseño del sistema, y el sistema receptor podría validar si todo el grupo de bits contiene un #par o impar de bits, y así se podría emitir una alerta en caso de ser incorrecto.</td></tr>
<tr><td colspan="2"> 

Paridad Par: sumar todos los 1, y si da impar es error</td><td> 

Paridad ImPar: sumar todos los 1, y si da impar es correcto</td></tr>
<tr><td></td><td></td></tr>
</table>

* Operaciones aritmetica +, -, *, :

Suma	**Resta**
0+0=0	0-0=0
0+1=1	0-1=1 (0 y llevo 1) o acarreo
1+0=1	1-0=1
1+1=10 (0 y llevo 1) o acarreo	1-1=0

Suma Binarias								
	1	**0**						0+0=0
		1	**0**				**BCD**	0+1=1
			1	**0**				1+0=1
				1	**0**		000=0	1+1=10 (0 y llevo 1) o acarreo
			1	1	1	1	001=1	
			1	1	1	1	010=2	
			1	1	1	1	011=3	1+1+1+1=100 (0 y llevo 10) acarreo
		+	1	1	1	1	100=4	1+1+1+1+1=101 (1 y llevo 10) acarreo
	1	1	1	1	0	0	101=5	
							110=6	
							111=7	

Restas binarias												
											BCD	0-0=0
												10-1=1 (1 de acarreo q son 2 decimal y 2-1=1)
		1	1				1				000=0	1-0=1
		1	$^{1}0$	$^{1}0$	1	1	1	$^{1}0$	1	1	001=1	1-1=0
	-	0	1	1	1	0	0	1	0	1	**010=2**	
		0	0	1	0	1	0	1	1	0	011=3	
											100=4	
											101=5	
											110=6	
											111=7	